
React Hooks

Ms. QuynhTNN

Objectives

- What React hooks are
- Use Basic React Hook for your component: `useState`, `useEffect`, `useContext`
- Custom Hook



What React hooks are

Hooks

- Hooks are a new addition in React 16.8.
- They let us use state and other different React features from our components without writing a class.
- Can be able to use the built-in Hooks or combine them to our own.

State Hooks

State Hooks

- State lets a component “**remember**” information like user input.
- For example, a form component can use state to store the **input value**, while an image gallery component can use state to store the **selected image index**.
- **To add state** to a component, use one of these Hooks:
 - **useState** declares a state variable that you can update directly.
 - **useReducer** declares a state variable with the update logic inside a reducer function.

useState

- useState is a React Hook that lets you add a state variable to your component.

```
const [state, setState] = useState(initialState)
```

- Usage:
 - Adding state to a component
 - Updating state based on the previous state
 - Updating objects and arrays in state

Effect Hooks

Effect Hooks

- Effects let a component connect to and synchronize with external systems.
- This includes dealing with network, browser DOM, animations, widgets written using a different UI library, and other non-React code.

useEffect

- useEffect is a React Hook that lets you synchronize a component with an external system.

```
useEffect(setup, dependencies?)
```

- Usage:
 - Fetching data with Effects
 - Connecting to an external system
 - Specifying reactive dependencies

Context Hook

Context Hooks

- Context lets a component receive information from distant parents without passing it as props.
- For example, your app's top-level component can pass the current UI theme to all components below, no matter how deep.

useContext

- useContext is a React Hook that lets you read and subscribe to context from your component.

```
const value = useContext(SomeContext)
```

- Usage:
 - Passing data deeply into the tree
 - Updating data passed via context
 - Specifying a fallback default value

Custom Hook

Custom Hook

- Hooks are reusable functions.
- When you have component logic that needs to be used by multiple components, we can extract that logic to a custom Hook.
- Custom Hooks start with "use". Example: useFetch.

Exercises

Summary

- Understand React hooks are
- Use Basic React Hook for your component: `useState`, `useEffect`, `useContext`
- Custom Hook

